
Designing Rewards for Fast Learning

Henry Sowerby

Department of Computer Science
Brown University
Providence, RI 02912
henry_sowerby@brown.edu

Zhiyuan Zhou

Department of Computer Science
Brown University
Providence, RI 02912
zhouzy@brown.edu

Michael L. Littman

Department of Computer Science
Brown University
Providence, RI 02912
mlittman@cs.brown.edu

Abstract

To convey desired behavior to a Reinforcement Learning (RL) agent, a designer must choose a reward function for the environment, arguably the most important knob designers have in interacting with RL agents. Although many reward functions induce the same optimal behavior (Ng et al., 1999), in practice, some of them result in faster learning than others. In this paper, we look at how reward-design choices impact learning speed and seek to identify principles of good reward design that quickly induce target behavior. This reward-identification problem is framed as an optimization problem: Firstly, we advocate choosing state-based rewards that maximize the action gap, making optimal actions easy to distinguish from suboptimal ones. Secondly, we propose minimizing a measure of the horizon, something we call the “subjective discount”, over which rewards need to be optimized to encourage agents to make optimal decisions with less lookahead. To solve this optimization problem, we propose a linear-programming based algorithm that efficiently finds a reward function that maximizes action gap and minimizes subjective discount. We test the rewards generated with the algorithm in tabular environments with Q-Learning, and empirically show they lead to faster learning. Although we only focus on Q-Learning because it is perhaps the simplest and most well-understood RL algorithm, preliminary results with R-max (Brafman and Tennenholtz, 2000) suggest our results are much more general. Our experiments support three principles of reward design: 1) consistent with existing results, penalizing each step taken induces faster learning than rewarding the goal. 2) When rewarding subgoals along the target trajectory, rewards should gradually increase as the goal gets closer. 3) Dense reward that’s nonzero on every state is only good if designed carefully.

Keywords: Reward design, Q-Learning, Machine Learning as Programming, Inverse Reinforcement Learning, Interactive Reinforcement Learning

Acknowledgements

We are grateful to David Abel and his reward research group at Google DeepMind for discussions and feedback, as well as NSF, DARPA, and ONR for funding support.

Problem Setting

We formulate the RL problem as a Markov Decision Process, modeling the environment in terms of states S , actions A , reward function $R(s)$, discount factor γ , and transition function $T(s, a, s')$. Reward functions are often defined on state–action pairs (Puterman, 1994), but we limit ourselves to the simpler case in which a reward function depends only on the states (Russell and Norvig, 1994). To further simplify reward specification, states are described in terms of a vector of features $F(s, i)$ and rewards are computed as a linear combination of these features (Ng and Russell, 2000). We seek to encourage, via rewards on state features, a target policy π^+ (Abel et al., 2021) mapping states to actions. An optimal policy π_R^* is one that maximizes the cumulative discounted expected reward from all states. We say a reward function R is *correct* if $\pi_R^* = \pi^+$.

To evaluate the ease of learning for a given correct reward function, we count the number of discrete timesteps where the learner’s preference matches the target policy. We stray away from the usual benchmark of cumulative reward achieved because reward is misleading in our case; designing a reward function that has across-the-board higher reward is not necessarily better at the task of encouraging the target behavior. Specifically, we define

$$\text{correct actions} = \sum_{t=1}^T \mathbb{I}[\text{argmax}_a Q(s_t, a) = \pi^+(s_t)].$$

We start by examining the diversity of correct reward functions for a given target policy. Consider the grid world example from Russell and Norvig (1994), described in Figure 1. To assess desirable aspects of different reward functions, we sample a collection of random reward functions that set $R(s) \in [-1, 1]$ for each state $s \in S$. Of the approximately 10,000,000 randomly-generated reward functions, 5,000 (0.05%) were correct, and we ran Q-Learning on these reward functions for 10,000 steps. The right margin of Figure 2 shows the distribution of cumulative number of correct actions these rewards induce. The distribution shows that although each of these reward functions produce the target policy when fully optimized, they differ significantly in the speed with which Q-Learning aligns itself with this target policy.

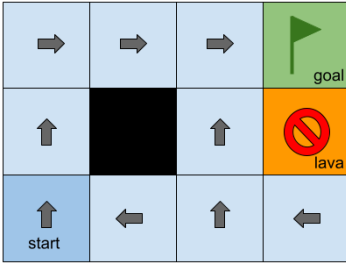


Figure 1: Russell/Norvig grid: The agent tries to get to the goal while avoiding the lava. At each step, the agent can choose to move in four cardinal directions, each of which has probability 0.8 to transition in that direction, and probability 0.1 to slip into the two orthogonal directions. The target policy is illustrated with arrows.

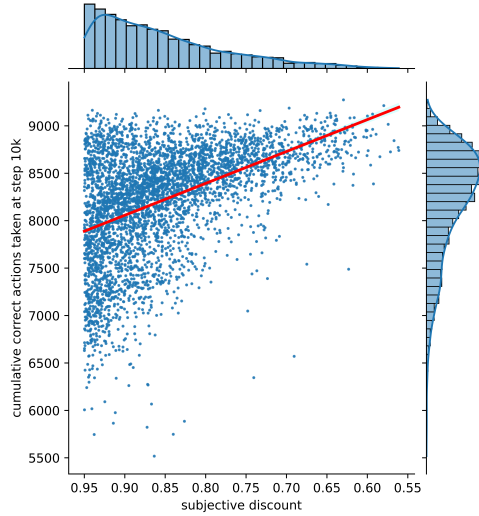


Figure 2: Cumulative correct actions of random reward functions vs. their subjective discount, Russell/Norvig grid, averaged over 1,000 runs. Regression in red.

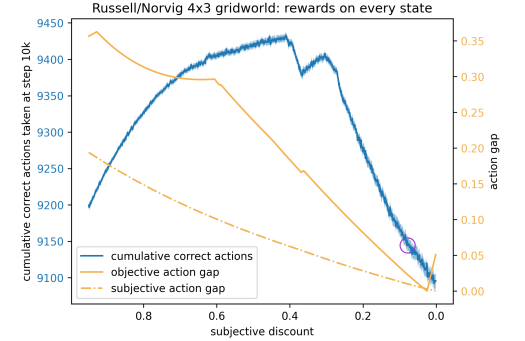


Figure 3: Learning performance (with 99% confidence interval averaged over 5,000 runs) of linear-program-designed rewards and their action gap. The objective action gap is computed at $\gamma = 0.95$, and the subjective action gap at $\tilde{\gamma}$. The purple circle marks the cumulative correct actions at threshold $\delta = 0.01$.

Subjective Discount

In trying to understand the properties of reward functions that lead to faster learning, we leverage the following insight: The convergence of algorithms like Q-Learning and R-Max and even value iteration depends on the discount factor. Smaller discount factors lead to faster convergence.

In our setting, we cannot simply adopt a smaller discount factor because we consider it to be part of the environment. As such, we adopt an idea from Jiang et al. (2015) and differentiate between two kinds of discount factors: the objective discount γ of the environment and a *subjective discount* $\tilde{\gamma}$, the latter of which is a property of the reward function. γ is used by the Q-Learning agent to estimate returns during learning, and $\tilde{\gamma}$ to design rewards.

Given a reward function R , we define π_R^γ to be the policy of an agent optimizing R in an environment with discount γ . The subjective discount for reward function R and target policy π^+ is the smallest $\tilde{\gamma}$ such that $\pi_R^{\tilde{\gamma}} = \pi^+, \forall \gamma' \in [\tilde{\gamma}, \gamma]$. That is, we want to know how small the discount factor could get while still encouraging the target policy. In our implementation, we use a binary search to efficiently compute the subjective discount.

Even though the learning agent does not have direct access to $\tilde{\gamma}$, it still seems to be a useful marker for good reward functions. Returning to Figure 2, the x-axis plots the subjective discount against the cumulative number of correct actions that match our target policy. Although the two measures are not perfectly correlated, low subjective discounts are reliable estimators of fast learning.

Reward Construction via Linear Programming

To aid our exploration of the relationship between subjective discount and rewards for fast learning, we created an efficient algorithm for identifying correct reward functions with minimal subjective discounts. Whereas our random reward-function search was able to identify reward functions with subjective discounts of around 0.55 for the Russell/Norvig grid, our linear-programming-based optimization process can push this value down to effectively zero. The generated reward for step cost, goal reward, and lava penalty, respectively, was $(-0.0223, +0.6119, -1)$, which outperforms the original rewards $(-0.04, +1, -1)$, with 88,364 vs. 83,666 correct actions, after 100k steps, averaged over 100 runs.

We make use of the discounted expected feature expectation (Abbeel and Ng, 2004) $D(s, i)$ and state-action feature expectations $D_a(s, i)$ for state s , action a , and feature i . They are defined using the feature value $F(s, i)$ and target policy π^+ at discount γ :

$$D(s, i) = F(s, i) + \gamma \sum_{s'} T(s, \pi^+(s), s') \cdot D(s', i)$$

$$D_a(s, i) = F(s, i) + \gamma \sum_{s'} T(s, a, s') \cdot D(s', i)$$

Similarly, we also define the same quantities $\tilde{D}(i, s)$ and $\tilde{D}_a(s, i)$ using subjective discount $\tilde{\gamma}$. It has been shown (Syed et al., 2008) that the state value function and Q-value of the target policy can be expressed as $V^{\pi^+}(s) = \sum_i D(s, i) \cdot R(i)$ and $Q^{\pi^+}(s, a) = \sum_i D_a(s, i) \cdot R(i)$, where $R(i)$ is the reward function based on the feature i .

The goal of our optimizer is to construct a reward function that induces the target policy π^+ at the objective discount *and* the lowest possible subjective discount. Our linear program is then formulated as:

$$\begin{aligned} & \text{choose } \delta, R(i) \text{ to maximize } \delta \\ & \text{subject to } \sum_i D(s, i) \cdot R(i) \geq \sum_i D_a(s, i) \cdot R(i) + \delta \\ & \quad \sum_i \tilde{D}(s, i) \cdot R(i) \geq \sum_i \tilde{D}_a(s, i) \cdot R(i) + \delta \\ & \quad -1 \leq R(i) \leq 1 \\ & \quad \forall i, s \in S, a \in A \setminus \{\pi^+(s)\} \end{aligned}$$

In words, choose the reward function such that, for each state, the target policy’s action in that state is at least δ better than any other action (for both the objective and subjective discount factors). The role of the δ variable here is to make sure that the target policy’s action is truly better ($\delta > 0$) than the other actions.

Next, we more closely evaluate the rewards constructed. For a range of $\tilde{\gamma} \in [0, 1]$, we ran the linear program to construct a range of reward functions, one for each $\tilde{\gamma}$. Figure 3 plots the cumulative correct actions taken in 10,000 steps for these rewards. The result shows that the best learning performance is achieved at an intermediate value of subjective discount, and we hypothesize that the reason is a kind of policy optimization regularization (Jiang et al., 2015).

Action Gap

Although the maximization objective δ was included as a mathematical convenience, it measures a property that has been recognized as significant in the literature. The difference between the value of the optimal action and the second best action is known as the *action gap* (Farahmand, 2011). It is commonly believed that a large action gap is beneficial for learning, mitigating effects of estimation errors (Lehnert et al., 2018), improving policies derived from Q-values (Bellemare et al., 2015), and achieving faster convergence rates (Farahmand, 2011). To hedge against action gap being optimized to 0 as subjective discount decreases, we add the constraint $\delta \geq 0.01$.

Principles of Good Reward Design

Advice: Penalize Steps

In a first-of-its kind result, Koenig and Simmons (1993) analyzed the time complexity of Q-Learning in a deterministic goal-based task, where they considered two two-featured correct reward functions: $(1, 0)$ and $(0, -1)$. The first, the *goal-reward* representation, provides the agent with a reward of $+1$ when the goal is reached and 0 while en route. The second, the *action-penalty* representation, penalizes the agent for each step taken, ending when the agent reaches the goal.

From a zero-initialized Q-function, they argued that the time to learn a good behavior in the goal-reward representation can grow exponentially with the size of the state space, while in the action-penalty representation $O(|A||S|^2)$ steps suffice. Essentially, the zero-initialization and the zero rewards means the agent has no guidance and explores randomly until the goal is encountered by accident. With action penalties, the agent tries to avoid actions it has already taken, resulting in much more directed exploration.

We replicate this finding experimentally in a simple 60-state chain MDP with objective discount 0.95 . It is a one-dimensional environment where the agent starts at the leftmost state, and needs to take rightward actions to arrive at the goal state on the opposite side. The two available actions transition the agent deterministically left or right.

We compare the performance of the two reward functions just described with one that our linear program generates. For this environment, our linear program chooses $(1, -1)$, which we call “combo” as it combines the penalty and the reward.

At the objective discount of 0.95 , the action gaps for the three reward functions are approximately 0.0024 (goal reward), 0.0485 (action penalty), and 0.0509 (combo). Since we use an action-gap threshold of 0.01 , goal reward doesn’t have a well defined subjective discount, while action penalty achieves a subjective discount of 0.9249 , and combo 0.9238 , approximately. The subjective discounts suggest that combo and action penalty will be similar, and both better than goal reward. Figure 4 confirms this hypothesis.

Advice: Reward Subgoals

In the context of hierarchical reinforcement learning (Parr and Russell, 1998), long-horizon tasks are broken into smaller tasks. Commonly, policies for the subtasks have their own pseudorewards that encourage subtask completion. In the context of a long-horizon goal, how should rewards be assigned to subtasks to encourage fast and accurate learning?

In naturally occurring problems, subtasks can break up goal-seeking sequences into heterogeneous chunks. In the simplified setting we study here, subtask completion states are spaced evenly along the 60-state chain and each has its own feature (and therefore can receive its own reward value).

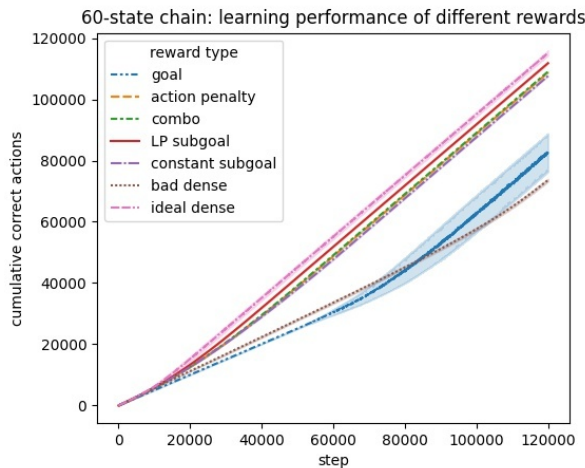


Figure 4: Comparison of reward functions on the 60-state chain.

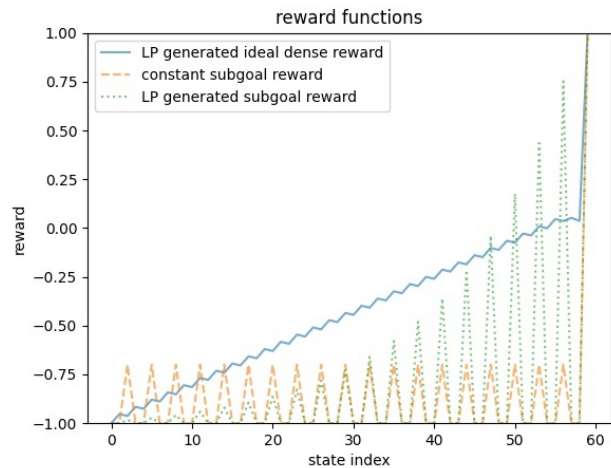


Figure 5: Two reward functions that place rewards on intermediate subgoals

We consider subgoal states placed every three states along the chain. The natural design choice would be to reward each of the resulting 19 subgoals the same reward value r , the goal $+1$, and all other states a step penalty -1 , as suggested by the last section. However, it is not immediately obvious how to arrive at a value for r such that the more frequently

spaced rewards induce the target policy. One example that we have found is $r = -0.7$, a small advantage over the -1 rewarded non-subgoal states.

We compared the constant subgoal reward above to a subgoal reward profile generated by our linear program. The reward functions are shown in Figure 5. The constant subgoal reward has a subjective discount of 0.9510, while the subgoal profile has a much lower subjective discount of 0.8232. The learning results are in Figure 4.

From Figure 4, we observe significant differences in learning time; the subgoal profile reward learns significantly faster than the constant subgoal reward, and, the goal-only “combo” reward from the last section is included, which actually learns faster than the constant subgoal reward. This result suggests that only appropriate subgoal rewards speed learning, while inappropriate ones may actually lead to slower learning.

We find that, when defining subgoal rewards, it helps to gradually increase rewards as the agent gets closer to the goal state. This design helps counteract the effect of discounting, but also continually spurs the agent forward, much like an annual salary raise is considered to be a good motivator in the commercial sector.

Advice: Use Dense Rewards

It is widely recognized that the kind of sparse rewards that come from rewarding only the goal is inferior to “dense” rewards like the subgoal rewards discussed in the previous section (Smart and Kaelbling, 2002).

Here, we look at the limiting case where every state has its own reward value. Figure 5 shows the reward function found by the linear program. It has a subjective discount of 0.2128. Organizing rewards in a jagged shape lets the reward function have the largest possible state-by-state increase while still remaining in the required -1 to $+1$ range. Notice that the intermediate rewards are all negative in value—that’s to prevent the agent from finding the intermediate rewards more attractive than reaching the goal.

Figure 4 compares the Q-Learning performance on this LP-produced dense reward to the sparse rewards from the previous section. We also included a second fully dense reward function selected to have a low action gap of near 0, which performs much worse. An important lesson here is that sparse rewards are indeed difficult to learn from, but not all dense rewards are equally good.

References

- Abbeel, P. and Ng, A. Y. (2004). Apprenticeship learning via inverse reinforcement learning. In *Proceedings of the Twenty-First International Conference on Machine Learning*, page 1.
- Abel, D., Dabney, W., Harutyunyan, A., Ho, M. K., Littman, M. L., Precup, D., and Singh, S. (2021). On the expressivity of Markov reward. arXiv preprint arXiv:2111.00876.
- Bellemare, M. G., Ostrovski, G., Guez, A., Thomas, P. S., and Munos, R. (2015). Increasing the action gap: New operators for reinforcement learning.
- Brafman, R. I. and Tennenholtz, M. (2000). A near-optimal polynomial time algorithm for learning in certain classes of stochastic games. *Artificial Intelligence*, 121(1–2):31–47.
- Farahmand, A.-m. (2011). Action-gap phenomenon in reinforcement learning. In *Advances in Neural Information Processing Systems*, pages 172–180.
- Jiang, N., Kulesza, A., Singh, S., and Lewis, R. (2015). The dependence of effective planning horizon on model accuracy. In *Proceedings of AAMAS*, pages 1181–1189.
- Koenig, S. and Simmons, R. G. (1993). Complexity analysis of real-time reinforcement learning. In *Proceedings of the Eleventh National Conference on Artificial Intelligence*, pages 99–105, Menlo Park, CA. AAAI Press/MIT Press.
- Lehnert, L., Laroché, R., and van Seijen, H. (2018). On value function representation of long horizon problems. In *AAAI Conference on Artificial Intelligence*.
- Ng, A. Y., Harada, D., and Russell, S. (1999). Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the Sixteenth International Conference on Machine Learning*, pages 278–287.
- Ng, A. Y. and Russell, S. (2000). Algorithms for inverse reinforcement learning. In *International Conference on Machine Learning*, pages 663–670.
- Parr, R. and Russell, S. (1998). Reinforcement learning with hierarchies of machines. In *Advances in Neural Information Processing Systems: Proceedings of the 1997 Conference*.
- Puterman, M. L. (1994). *Markov Decision Processes—Discrete Stochastic Dynamic Programming*. John Wiley & Sons, Inc., New York, NY.
- Russell, S. J. and Norvig, P. (1994). *Artificial Intelligence: A Modern Approach*. Prentice-Hall, Englewood Cliffs, NJ.
- Smart, W. D. and Kaelbling, L. P. (2002). Effective reinforcement learning for mobile robots. In *Proceedings 2002 IEEE International Conference on Robotics and Automation*, volume 4, pages 3404–3410. IEEE.
- Syed, U., Bowling, M., and Schapire, R. E. (2008). Apprenticeship learning using linear programming. In *Proceedings of the 25th International Conference on Machine Learning, ICML ’08*, page 1032–1039, New York, NY, USA. Association for Computing Machinery.